

DATALENS – APPROACH, DECISIONS, WHAT'S NEXT

THE PROBLEM BEHIND THE PROBLEM

Schema in, model out, run the SQL, render — that takes an hour and demos well on a clean CSV. It fails in front of a customer in specific ways, each producing a **wrong answer that looks right**: an invented column; `status = 'Active'` when the file says `ACTIVE`, returning zero rows reported as the number 0; a join on an unrelated key that inflates every total; `01/02/2024` read as January when the file means February.

So I treated *trustworthiness*, not SQL generation, as the engineering problem.

THE ORGANISING DECISION

The model decides what to compute. SQLite decides what the number is.

The model emits a query, a database executes it against the real rows, and only then does the model see the result and write a sentence around it. No figure in the UI comes from token prediction — arithmetic hallucination isn't reduced, it's structurally impossible. The remaining failure mode is a wrong *query*, which is inspectable, so the SQL sits one click under every answer.

KEY DECISIONS

SQLite compiled to WebAssembly, in the browser. The datasets here are payroll and headcount, so "your rows never leave the tab" is a real objection answered rather than a policy promise — only the schema crosses the network. It also makes the server a pure function, which is why this deploys as one project with no database or session store. The cost is a ceiling around 100k rows.

Grounding beats prompting. Columns are profiled at upload: exact names, types, ranges, and the *actual value list* for low-cardinality columns. Join keys are found by measuring how much of one column's values genuinely appear in another. Handed evidence, the model has no reason to guess — which is what makes cross-file questions work without the user explaining their files.

The guard is code, not an instruction. Model output is untrusted input: single statement, `SELECT` only, with literals and comments blanked before keyword matching. Safety doesn't change when the model does.

Ambiguity is disclosed, not resolved silently. Date direction is settled from evidence; where nothing settles it, the assumption is stated in the UI. The model may also decline and ask what you meant.

TESTING

The parts that decide correctness are pure and carry ~118 assertions. An end-to-end run checks results against figures computed independently from the CSV text, and a browser suite walks each acceptance criterion against the live deployment. That last one found four defects typecheck, lint and unit tests had all waved through — including a WASM binary that 404'd, so SQLite never loaded.

WHAT I'D BUILD NEXT

Close the remaining correctness gaps: a verification pass asking whether the result actually answers the question, and a check that warns when a join multiplies rows past the larger input — the most damaging silent error there is.

Answer differently depending on who is asking. In an HR system this is the real problem. "Average salary by department" is reasonable from a CHRO and a data breach from a team lead. That means scoping the query before execution rather than filtering results after, and saying *why* a number is scoped instead of silently narrowing it.

Point it at live systems instead of uploads. Uploading an export is how you demo this; connecting to the system of record is how anyone uses it. Profiling and querying are already separate, so a live schema slots into the same shape — the guard and the grounding don't change.

Move the engine server-side past the browser's ceiling. In-browser SQLite is right for files a person can upload and wrong at millions of rows ([ADR 0001](#)). The model/database boundary is unchanged by that move, which makes it a swap rather than a rewrite.

Then the smaller things: follow-up questions so "and by location?" works, per-column type overrides, and Web Worker ingest.

Seven ADRs in [docs/adr/](#) record each decision with its costs stated plainly.